

Presentation of References 1

Shanik Hubert

Contents

Precoding Design for Multi-user MIMO Systems with
Delay-Constrained and Delay-Tolerant Users

Adaptive Finite Blocklength for Ultra-Low Latency in Wireless
Communications

Comparison with our method

Paper 1

M. Kim, J. Park and J. Lee, "MU-MIMO Precoding Design in the Presence of Delay-Constrained Users," ICC 2021 - IEEE International Conference on Communications, Montreal, QC, Canada, 2021,

Introduction: DCTU-MIMO Precoding

Abstract

This paper studies precoding design for multi-user MIMO systems serving both delay-constrained and delay-tolerant users. The objective is to maximize the ergodic sum spectral efficiency while satisfying latency and reliability requirements under finite blocklength transmission. A generalized power iteration (GPI) based precoding method is used to efficiently handle the resulting non-convex optimization problem. Numerical results show that the proposed Delay-GPI and Infinite-GPI schemes outperform conventional precoding methods such as RZF and MRT, especially under different user weights and delay constraints.

System Overview

- ▶ A base station with N antennas serves two types of users:
 - ▶ **Delay-tolerant users:** require high spectral efficiency.
 - ▶ **Delay-constrained users:** require strict latency guarantees.
- ▶ A base station with N antennas serves:

K_t delay-tolerant users, K_s delay-constrained users

- ▶ Total user set:

$$\mathcal{K} = \mathcal{K}_t \cup \mathcal{K}_s$$

- ▶ The transmitted signal is precoded as:

$$\mathbf{x} = \sum_{k=1}^{K_t+K_s} \mathbf{u}_k s_k$$

where $\mathbf{u}_k$ is the precoding vector for user k .

- ▶ The received signal at user k is:

$$y_k = \mathbf{h}_k^H \mathbf{u}_k s_k + \sum_{i \neq k} \mathbf{h}_k^H \mathbf{u}_i s_i + z_k$$

- ▶ Objective: maximize delay-tolerant users' sum spectral efficiency while satisfying delay-constrained users' latency

- ▶ The paper solves this non-convex problem using a generalized power iteration method called **Delay-GPI**.

Spectral Efficiency Modeling

- ▶ The SINR of user k is:

$$\gamma_k = \frac{|\mathbf{h}_k^H \mathbf{u}_k|^2}{\sum_{i \neq k} |\mathbf{h}_k^H \mathbf{u}_i|^2 + \frac{\sigma_k^2}{P}}$$

- ▶ For delay-tolerant users, Shannon capacity is used:

$$R_k(\gamma_k) = \log_2(1 + \gamma_k), \quad k \in \mathcal{K}_t$$

- ▶ For delay-constrained users, finite blocklength rate is used:

$$R_k(\gamma_k) = \log_2(1 + \gamma_k) - \sqrt{\frac{V(\gamma_k)}{m}} Q^{-1}(\epsilon_k), \quad k \in \mathcal{K}_s$$

- ▶ Here, m is blocklength and ϵ_k is target error probability.
- ▶ The second term is the finite blocklength penalty, which reduces the achievable rate for low-latency users.

Rayleigh Quotient Reformulation

- ▶ A Rayleigh quotient is a quadratic matrix ratio of the form:

$$\frac{\mathbf{x}^H \mathbf{A} \mathbf{x}}{\mathbf{x}^H \mathbf{B} \mathbf{x}}$$

- ▶ Here:

$\mathbf{x}$ = optimization vector, $\mathbf{A}, \mathbf{B}$ = matrices

- ▶ In this paper, the optimization vector is the stacked precoder:

$$\mathbf{u} = [\mathbf{u}_1^H, \mathbf{u}_2^H, \dots, \mathbf{u}_{K_t+K_s}^H]^H$$

- ▶ Therefore, the Rayleigh quotient becomes:

$$\frac{\mathbf{u}^H \mathbf{A}_k \mathbf{u}}{\mathbf{u}^H \mathbf{B}_k \mathbf{u}}$$

- ▶ This quotient represents:

$$\frac{\text{signal} + \text{interference} + \text{noise}}{\text{interference} + \text{noise}} = 1 + \gamma_k$$

- ▶ This allows us to apply the GPI algorithm (as it makes it easier to reformulate into an eigenvalue problem formulation).

- ▶ Start with the SINR of user k :

$$\gamma_k = \frac{|\mathbf{h}_k^H \mathbf{u}_k|^2}{\sum_{i \neq k} |\mathbf{h}_k^H \mathbf{u}_i|^2 + \frac{\sigma_k^2}{P}}$$

- ▶ Hence:

$$1 + \gamma_k = \frac{\sum_{i=1}^{K_t+K_s} |\mathbf{h}_k^H \mathbf{u}_i|^2 + \frac{\sigma_k^2}{P}}{\sum_{i \neq k} |\mathbf{h}_k^H \mathbf{u}_i|^2 + \frac{\sigma_k^2}{P}}$$

- ▶ Define the stacked precoding vector:

$$\mathbf{u} = [\mathbf{u}_1^H, \mathbf{u}_2^H, \dots, \mathbf{u}_{K_t+K_s}^H]^H$$

$\mathbf{A}_k$ and $\mathbf{B}_k$

- ▶ Let:

$$K = K_t + K_s, \quad \mathbf{H}_k = \mathbf{h}_k \mathbf{h}_k^H$$

- ▶ The matrix $\mathbf{A}_k$ contains all received signal power terms:

$$\mathbf{A}_k = \text{diag}(\mathbf{H}_k, \mathbf{H}_k, \dots, \mathbf{H}_k) + \frac{\sigma_k^2}{P} \mathbf{I}_{NK}$$

- ▶ Therefore:

$$\mathbf{u}^H \mathbf{A}_k \mathbf{u} = \sum_{i=1}^K |\mathbf{h}_k^H \mathbf{u}_i|^2 + \frac{\sigma_k^2}{P}$$

= desired signal + interference + noise

- ▶ The matrix $\mathbf{B}_k$ removes the desired signal block of user k :

$$\mathbf{B}_k = \mathbf{A}_k - \text{diag}(0, \dots, \mathbf{H}_k, \dots, 0)$$

- ▶ Hence:

$$\mathbf{u}^H \mathbf{B}_k \mathbf{u} = \sum_{i \neq k} |\mathbf{h}_k^H \mathbf{u}_i|^2 + \frac{\sigma_k^2}{P}$$

= interference + noise

► Thus:

$$\frac{\mathbf{u}^H \mathbf{A}_k \mathbf{u}}{\mathbf{u}^H \mathbf{B}_k \mathbf{u}} = \frac{\text{signal} + \text{interference} + \text{noise}}{\text{interference} + \text{noise}} = 1 + \gamma_k$$

- ▶ For delay-tolerant users, Shannon capacity is used directly since they are assumed to have long blocklength:

$$R_k(\mathbf{u}) = \log_2 \left(\frac{\mathbf{u}^H \mathbf{A}_k \mathbf{u}}{\mathbf{u}^H \mathbf{B}_k \mathbf{u}} \right), \quad k \in \mathcal{K}_t$$

- ▶ For delay-constrained users, finite blocklength coding is used:

$$R_k(\gamma_k) = \log_2(1 + \gamma_k) - \sqrt{\frac{V(\gamma_k)}{m}} Q^{-1}(\epsilon_k)$$

- ▶ Since this is difficult to optimize, the paper uses the lower bound(derivation is in the paper):

$$\tilde{R}_k(\mathbf{u}, \tilde{\gamma}_k) = \log_2 \left[\left(\frac{\mathbf{u}^H \mathbf{A}_k \mathbf{u}}{\mathbf{u}^H \mathbf{B}_k \mathbf{u}} \right)^{1-f_k(\tilde{\gamma}_k)} \right] - g_k(\tilde{\gamma}_k)$$

- ▶ Therefore, the Rayleigh quotient is used for both user types.

First-Order Optimality Condition (1): Lagrangian Setup

- ▶ The optimization problem is:

$$\begin{aligned} \max_{\mathbf{u}} \quad & \sum_{k=1}^{K_t} R_k(\mathbf{u}) \\ \text{s.t.} \quad & \tilde{R}_k(\mathbf{u}, \tilde{\gamma}_k) \geq \frac{D_s}{\delta_k}, \quad k \in \mathcal{K}_s. \end{aligned}$$

- ▶ Only sum rate of delay tolerant users are optimized, rate of delay constrained users just need to be enough to satisfy latency constraints.
- ▶ D_s is the data size of downlink signal, δ_k is latency of user k .
- ▶ Convert maximization into minimization:

$$\min_{\mathbf{u}} - \sum_{k=1}^{K_t} R_k(\mathbf{u})$$

- ▶ Introduce Lagrange multiplier λ_k for each delay-constrained user:

$$\lambda_k \geq 0, \quad k \in \mathcal{K}_s.$$

- Lagrangian:

$$\mathcal{L}(\mathbf{u}, \lambda) = - \sum_{k=1}^{K_t} R_k(\mathbf{u}) - \sum_{k=K_t+1}^{K_t+K_s} \lambda_k \left[\tilde{R}_k(\mathbf{u}, \tilde{\gamma}_k) - \frac{D_s}{\delta_k} \right].$$

- λ_k controls how strongly the latency constraint affects the precoder.

First-Order Optimality Condition (2): Substituting Rate Expressions

- Delay-tolerant user rate:

$$R_k(\mathbf{u}) = \log_2 \left(\frac{\mathbf{u}^H \mathbf{A}_k \mathbf{u}}{\mathbf{u}^H \mathbf{B}_k \mathbf{u}} \right)$$

- Delay-constrained lower-bound rate:

$$\tilde{R}_k(\mathbf{u}, \tilde{\gamma}_k) = \log_2 \left[\left(\frac{\mathbf{u}^H \mathbf{A}_k \mathbf{u}}{\mathbf{u}^H \mathbf{B}_k \mathbf{u}} \right)^{1-f_k} \right] - g_k.$$

- Substituting into the Lagrangian:

$$\begin{aligned} \mathcal{L} = & - \sum_{k=1}^{K_t} \log_2 \left(\frac{\mathbf{u}^H \mathbf{A}_k \mathbf{u}}{\mathbf{u}^H \mathbf{B}_k \mathbf{u}} \right) \\ & - \sum_{k=K_t+1}^{K_t+K_s} \lambda_k \left[\log_2 \left(\frac{\mathbf{u}^H \mathbf{A}_k \mathbf{u}}{\mathbf{u}^H \mathbf{B}_k \mathbf{u}} \right)^{1-f_k} - g_k - \frac{D_s}{\delta_k} \right]. \end{aligned}$$

- The terms g_k (rewritten as a constant while defining the lower bound) and $\frac{D_s}{\delta_k}$ do not depend on $\mathbf{u}$.

First-Order Optimality Condition (3): Grouping and Differentiation

- ▶ Using $\sum \log(x_k) = \log(\prod x_k)$, group all $\mathbf{u}$ -dependent terms as:

$$\mathcal{L}(\mathbf{u}, \lambda) = -\log_2 \phi(\mathbf{u}, \lambda) + \text{constant terms}$$

- ▶ where:

$$\phi(\mathbf{u}, \lambda) = \frac{\prod_{k=1}^{K_t} \mathbf{u}^H \mathbf{A}_k \mathbf{u} \prod_{k=K_t+1}^{K_t+K_s} (\mathbf{u}^H \mathbf{A}_k \mathbf{u})^{\lambda_k(1-f_k)}}{\prod_{k=1}^{K_t} \mathbf{u}^H \mathbf{B}_k \mathbf{u} \prod_{k=K_t+1}^{K_t+K_s} (\mathbf{u}^H \mathbf{B}_k \mathbf{u})^{\lambda_k(1-f_k)}}.$$

- ▶ First-order condition:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{u}^H} = 0$$

- ▶ Since:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{u}^H} = -\frac{1}{\phi(\mathbf{u}, \lambda) \ln 2} \frac{\partial \phi(\mathbf{u}, \lambda)}{\partial \mathbf{u}^H},$$

we get:

$$\frac{\partial \phi(\mathbf{u}, \lambda)}{\partial \mathbf{u}^H} = 0.$$

- Define:

$$\phi = \frac{N}{D}, \quad \frac{\partial N}{\partial \mathbf{u}^H} = \bar{\mathbf{A}}(\mathbf{u}, \lambda) \mathbf{u}, \quad \frac{\partial D}{\partial \mathbf{u}^H} = \bar{\mathbf{B}}(\mathbf{u}, \lambda) \mathbf{u}$$

- This leads to:

$$\bar{\mathbf{A}}(\mathbf{u}, \lambda) \mathbf{u} = \phi(\mathbf{u}, \lambda) \bar{\mathbf{B}}(\mathbf{u}, \lambda) \mathbf{u}.$$

- This is a generalized eigenvalue-type condition solved using Delay-GPI.

Delay-GPI Precoding Algorithm

- ▶ From the first-order condition:

$$\bar{\mathbf{A}}(\mathbf{u}, \lambda)\mathbf{u} = \phi(\mathbf{u}, \lambda)\bar{\mathbf{B}}(\mathbf{u}, \lambda)\mathbf{u}$$

- ▶ This resembles a generalized eigenvalue problem:

$$\mathbf{A}\mathbf{u} = \mu\mathbf{B}\mathbf{u}$$

- ▶ However, $\bar{\mathbf{A}}$ and $\bar{\mathbf{B}}$ depend on $\mathbf{u}$ itself.
- ▶ Therefore, the paper uses an iterative generalized power iteration method.
- ▶ Precoder update:

$$\mathbf{u}^{(j)} = \left[\bar{\mathbf{B}}(\mathbf{u}^{(j-1)}, \lambda^{(n)}) \right]^{-1} \bar{\mathbf{A}}(\mathbf{u}^{(j-1)}, \lambda^{(n)})\mathbf{u}^{(j-1)}$$

- ▶ Normalize:

$$\mathbf{u}^{(j)} = \frac{\mathbf{u}^{(j)}}{\|\mathbf{u}^{(j)}\|}$$

Delay-GPI Algorithm (2): Why Direct Eigenvalue Solution Fails

- ▶ If $\bar{\mathbf{A}}$ and $\bar{\mathbf{B}}$ were fixed, we could write:

$$\bar{\mathbf{B}}^{-1}\bar{\mathbf{A}}\mathbf{u} = \phi\mathbf{u}$$

- ▶ This is a normal eigenvalue problem:

$$\mathbf{M}\mathbf{u} = \phi\mathbf{u}, \quad \mathbf{M} = \bar{\mathbf{B}}^{-1}\bar{\mathbf{A}}$$

- ▶ But in this paper:

$$\bar{\mathbf{A}} = \bar{\mathbf{A}}(\mathbf{u}, \lambda), \quad \bar{\mathbf{B}} = \bar{\mathbf{B}}(\mathbf{u}, \lambda)$$

- ▶ So the matrices themselves depend on the unknown precoder $\mathbf{u}$.
- ▶ Therefore, the problem cannot be solved by one direct eigenvalue decomposition.
- ▶ The paper solves it iteratively using generalized power iteration.

Delay-GPI Algorithm (3): Precoder Update

- ▶ At iteration j , use the previous precoder $\mathbf{u}^{(j-1)}$ to construct:

$$\bar{\mathbf{A}}\left(\mathbf{u}^{(j-1)}, \lambda^{(n)}\right), \quad \bar{\mathbf{B}}\left(\mathbf{u}^{(j-1)}, \lambda^{(n)}\right)$$

- ▶ Then update the precoder as:

$$\mathbf{u}^{(j)} = \left[\bar{\mathbf{B}}\left(\mathbf{u}^{(j-1)}, \lambda^{(n)}\right) \right]^{-1} \bar{\mathbf{A}}\left(\mathbf{u}^{(j-1)}, \lambda^{(n)}\right) \mathbf{u}^{(j-1)}$$

- ▶ Normalize after each update:

$$\mathbf{u}^{(j)} = \frac{\mathbf{u}^{(j)}}{\|\mathbf{u}^{(j)}\|}$$

- ▶ Normalization satisfies the total transmit power constraint (which is assumed to be 1).

Delay-GPI: Feasibility and Convergence

- ▶ The Delay-GPI algorithm may not always find a feasible precoder.
- ▶ Feasibility depends on system parameters such as:

$$P, \quad \epsilon_k, \quad D_s, \quad \delta_k, \quad \mathbf{h}_k$$

- ▶ If a delay-constrained user has weak channel gain, the BS may need more transmit power to satisfy:

$$\tilde{R}_k(\mathbf{u}, \tilde{\gamma}_k) \geq \frac{D_s}{\delta_k}$$

- ▶ If the required latency is too strict or transmit power is too low, the constraint may be infeasible.
- ▶ In that case, the delay-constrained user is assumed to fail transmission and its contribution is not added to the ergodic sum spectral efficiency.
- ▶ The precoder update continues until the convergence criterion is satisfied:

$$\left\| \mathbf{u}^{(j)} - \mathbf{u}^{(j-1)} \right\| < \xi$$

- ▶ Here, ξ is a small tolerance value that controls when the iterative update stops.

Results - 1

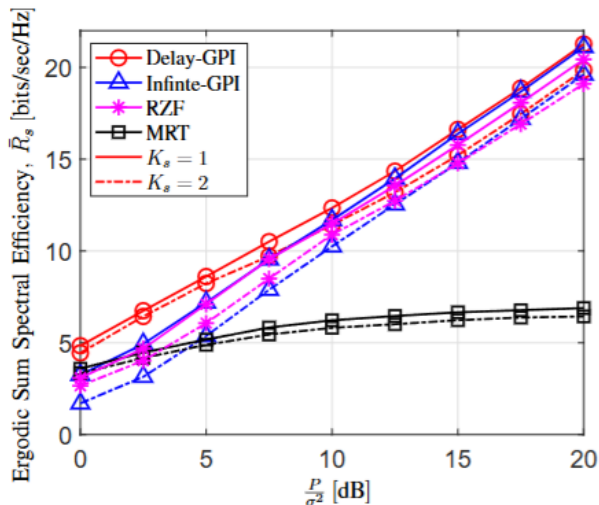


Figure 1: Ergodic sum spectral efficiency $\bar{R}_s$ as a function of $P/2$ with $m = 100$, $N = 8$ and $K_t = 3$ for different values of K_s . In the case of $K_s = 1$, $\delta_4 = 250$. When $K_s = 2$, $\delta_4 = 250$ and $\delta_5 = 450$.

Results - 2

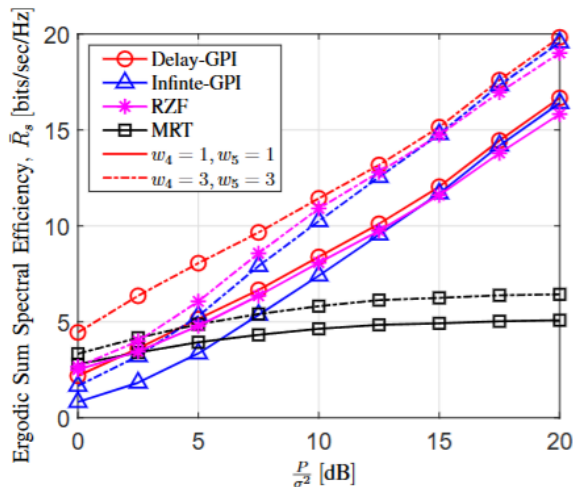


Figure 2: $\frac{P}{\sigma^2}$ with $m = 300$, $N = 8$, $K_t = 3$, $K_s = 2$, $\delta_4 = 250$, $\delta_5 = 450$, $\tilde{\gamma}_4 = 2.38$, and $\tilde{\gamma}_5 = 1.35$ for different values of w_k . $w_1, w_2, w_3 = 1$

Inference from Result 1: Effect of Number of Delay-Constrained Users

- ▶ Result 1 compares the ergodic sum spectral efficiency for:

$$K_s = 1 \quad \text{and} \quad K_s = 2$$

with fixed (N is number of transmit antenna):

$$N = 8, \quad K_t = 3.$$

- ▶ Result:

$$K_s \uparrow \Rightarrow \bar{R}_s \downarrow$$

- ▶ Reason: when more delay-constrained users are present, more transmit power must be reserved to satisfy:

$$\tilde{R}_k(\mathbf{u}, \tilde{\gamma}_k) \geq \frac{D_s}{\delta_k}.$$

- ▶ Therefore, less power remains for users with better channel gain.
- ▶ Delay-GPI still performs better because it explicitly considers finite blocklength latency constraints, while Infinite-GPI assumes:

$$m = \infty$$

during precoder design

Inference from Result 2: Effect of Delay-Constrained User Weights

- ▶ Result 2 compares different weights for delay-constrained users:

$$w_4 = w_5 = 1 \quad \text{and} \quad w_4 = w_5 = 3.$$

- ▶ Larger weights increase the contribution of delay-constrained users in:

$$R_s = \sum_{k=1}^{K_t} w_k R_k(\mathbf{u}) + \sum_{k=K_t+1}^{K_t+K_s} p_{c,k} w_k \frac{D_s}{\delta_k}.$$

- ▶ Result:

$$w_4, w_5 \uparrow \Rightarrow \text{Delay-GPI advantage becomes clearer.}$$

- ▶ When $w_4 = w_5 = 1$, the gap between Delay-GPI and other methods decreases.
- ▶ At very low power, MRT/RZF may appear slightly better because Delay-GPI can fail to find a feasible precoder.
- ▶ However, Delay-GPI is more meaningful because it is designed to satisfy the latency constraints of delay-constrained users.

Paper 2

W. Cheng, Y. Xiao, S. Zhang and J. Wang, “Adaptive Finite Blocklength for Ultra-Low Latency in Wireless Communications,” in IEEE Transactions on Wireless Communications, vol. 21, no. 6, pp. 4450-4463, June 2022,

Introduction

Paper: Adaptive Finite Blocklength for Ultra-Low Latency in Wireless Communications

Main problem:

- ▶ Future 6G systems require ultra-low end-to-end delay for autonomous vehicles, factory automation, remote control, and real-time machine–human interaction.
- ▶ Traditional LTE and 5G NR use fixed transmission time intervals (TTIs).
- ▶ The paper argues that fixed TTI is not always optimal for finite blocklength transmission.

Main idea:

Adapt the blocklength ie: TTI to minimize delay.

System Model

- ▶ End-to-end delay:

$$D_{\text{E2E}} = D_{\text{queue}} + D_{\text{trans}} + D_{\text{proc}} + D_{\text{prop}}$$

- ▶ Paper focuses on:

$$D_{\text{queue}} + D_{\text{trans}}$$

- ▶ One frame = one TTI(transmission time interval).
- ▶ Blocklength:

$$n = TB$$

$$T = \text{frame duration}, \quad B = \text{bandwidth}$$

- ▶ With fixed B :

$$n \downarrow \Rightarrow T \downarrow \Rightarrow D_{\text{trans}} \downarrow$$

Finite Blocklength Rate and Queue Service

- ▶ Finite blocklength achievable rate:

$$R(n) \approx \log_2(1 + P|h|^2) - \sqrt{\frac{V}{n}} Q^{-1}(\epsilon) \log_2 e$$

$$V = 1 - (1 + P|h|^2)^{-2}$$

- ▶ Information bits served in frame i :

$$\text{served bits}_i = R(n_i)n_i$$

- ▶ If each packet has a bits:

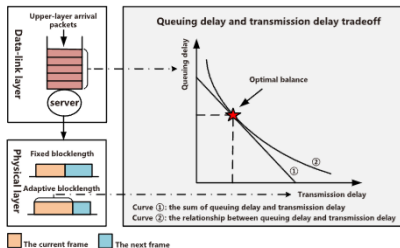
$$\text{served packets}_i = \frac{R(n_i)n_i}{a}$$

- ▶ Shorter blocklength reduces rate:

$$n_i \downarrow \Rightarrow R(n_i) \downarrow \Rightarrow R(n_i)n_i \downarrow \Rightarrow D_{\text{queue}} \uparrow$$

Adaptive blocklength balances D_{trans} and D_{queue} .

Blocklength Tradeoff



- ▶ Shorter blocklength means shorter frame duration: $n = TB$ where T is transmission time interval(TTI) and B is bandwidth(rate).
- ▶ Therefore, transmission delay decreases: $n \downarrow \Rightarrow D_{\text{trans}} \downarrow$
- ▶ But in finite blocklength transmission, shorter blocklength reduces achievable rate: $n \downarrow \Rightarrow R(n) \downarrow$
- ▶ Lower rate means fewer packet bits per frame: served bits = $R(n)n$
- ▶ Therefore, packets wait longer in the buffer:

$$R(n)n \downarrow \Rightarrow D_{\text{queue}} \uparrow$$

Single-User Delay Model

$$\mathbf{n} = (n_1, n_2, \dots, n_L)$$

$$D_p = d_p + \sum_{j=1}^{I(p)-1} n_j + n_{I(p)} = d_p + \sum_{j=1}^{I(p)} n_j$$

- ▶ N_p : number of packets
- ▶ L : number of adaptive frames
- ▶ $I(p)$: frame that transmits packet p
- ▶ d_p : previous waiting time

Single-User Delay Model

- ▶ The user has N_p short packets waiting in the buffer.
- ▶ These packets are transmitted over L adaptive frames:

$$\mathbf{n} = (n_1, n_2, \dots, n_L)$$

- ▶ n_i is the blocklength of frame i .
- ▶ If packet p is transmitted in frame $I(p)$, it must wait for all previous frames.

$$D_p = d_p + \sum_{j=1}^{I(p)-1} \frac{n_j}{B} + \frac{n_{I(p)}}{B} = d_p + \sum_{j=1}^{I(p)} \frac{n_j}{B}$$

- ▶ d_p : waiting time already accumulated from the previous period.
- ▶ $\sum_{j=1}^{I(p)-1} \frac{n_j}{B}$: queuing delay due to earlier frames.
- ▶ $\frac{n_{I(p)}}{B}$: transmission delay of the frame carrying packet p .

Single-User Optimization Problem

$$\begin{aligned} \mathbf{F}_1 : \quad & \min_{\mathbf{n}} \quad \frac{1}{N_p a} \sum_{i=1}^L R(n_i) n_i \sum_{j=1}^i n_j \\ \text{s.t.} \quad & \sum_{i=1}^L R(n_i) n_i = N_p a, && \text{data-completion constraint} \\ & \sum_{i=1}^L n_i \leq T_{\max}, && \text{period-duration constraint} \\ & n_i \geq 0, \quad i = 1, 2, \dots, L, && \text{valid blocklength constraint} \end{aligned}$$

- ▶ The objective represents the **average delay of all packets** served across L frames.
- ▶ In frame i , the number of transmitted information bits is:

$$R(n_i) n_i$$

- ▶ Since each packet has a bits, the number of packets served in frame i is: $\frac{R(n_i)n_i}{a}$
- ▶ Packets served in frame i wait until frame i finishes, so their delay is: $\sum_{j=1}^i n_j$
- ▶ Therefore, the total delay contributed by frame i is: $\frac{R(n_i)n_i}{a} \sum_{j=1}^i n_j$
- ▶ Summing over all frames and dividing by N_p gives the average packet delay:

$$\frac{1}{N_p a} \sum_{i=1}^L R(n_i)n_i \sum_{j=1}^i n_j$$

- ▶ Minimizing this chooses the blocklengths n_i that make the average packet delay as small as possible.
- ▶ Since the paper normalizes bandwidth, $B = 1$, the delay can be written directly using n_i .
- ▶ Hence n_i appears both as blocklength and as the frame-duration term in the delay expression.

Problem Transformation

$$\begin{aligned} \mathbf{F}_1 : \quad & \min_{\mathbf{n}} \quad \frac{1}{N_p a} \sum_{i=1}^L R(n_i) n_i \sum_{j=1}^i n_j \\ & \text{s.t.} \quad \sum_{i=1}^L R(n_i) n_i = N_p a, \\ & \quad \quad \sum_{i=1}^L n_i \leq T_{\max}, \quad n_i \geq 0 \end{aligned}$$

- ▶ The equality constraint is difficult because $R(n_i)$ is nonlinear in n_i .
- ▶ The paper moves this constraint into the objective using a penalty term:

$$\beta \left(\sum_{i=1}^L R(n_i) n_i - N_p a \right)^2$$

- ▶ This penalizes solutions that do not transmit all packet bits.

Penalty-Based Optimization

$$\begin{aligned} \mathbf{F}_2 : \quad & \min_{\mathbf{n}} \quad \frac{1}{N_{pa}} \sum_{i=1}^L R(n_i) n_i \sum_{j=1}^i n_j \\ & + \beta \left(\sum_{i=1}^L R(n_i) n_i - N_{pa} \right)^2 \\ \text{s.t.} \quad & \sum_{i=1}^L n_i \leq T_{\max}, \\ & n_i \geq 0, \quad i = 1, 2, \dots, L \end{aligned}$$

FaBo: Rewriting the Rate Term

Finite blocklength rate:

$$R(n_i) = \log_2(1 + P|h|^2) - \sqrt{\frac{V}{n_i}} Q^{-1}(\epsilon) \log_2 e$$

Define:

$$A_1 = \log_2(1 + P|h|^2)$$

$$A_2 = \sqrt{V} Q^{-1}(\epsilon) \log_2 e$$

Then:

$$R(n_i) = A_1 - \frac{A_2}{\sqrt{n_i}}$$

$$R(n_i)n_i = A_1 n_i - A_2 \sqrt{n_i}$$

- ▶ $A_1 n_i$: Shannon-like service part.
- ▶ $-A_2 \sqrt{n_i}$: finite-blocklength penalty part.
- ▶ This square-root term is what makes the objective difficult.

FaBo: Splitting the objective $G(\mathbf{n})$ and $H(\mathbf{n})$

$$F(\mathbf{n}) = G(\mathbf{n}) + H(\mathbf{n}) + \beta N_p^2 a^2$$

- ▶ $G(\mathbf{n})$ is the convex part.
- ▶ It contains linear and quadratic terms in n_i :

$$n_i, \quad n_i n_j, \quad \left(\sum_i n_i \right)^2$$

- ▶ $H(\mathbf{n})$ is the non-convex part.
- ▶ It contains square-root terms caused by finite blocklength:

$$\sqrt{n_i}, \quad \sqrt{n_i} n_j, \quad \sqrt{n_i} \sqrt{n_j}$$

- ▶ The constant $\beta N_p^2 a^2$ does not affect the optimizer.
- ▶ Therefore, FaBo solves only:

$$G(\mathbf{n}) + H(\mathbf{n})$$

FaBo: Augmented Lagrangian

For $\mathbf{F}_3$, the augmented Lagrangian is:

$$\mathcal{L}(\mathbf{x}, \mathbf{y}, \boldsymbol{\lambda}) = G(\mathbf{x}) + H(\mathbf{y}) - \boldsymbol{\lambda}^T (\mathbf{x} - \mathbf{y}) + \frac{\rho}{2} \|\mathbf{x} - \mathbf{y}\|^2$$

- ▶ $\boldsymbol{\lambda}$: Lagrange multiplier vector for constraint $\mathbf{x} - \mathbf{y} = 0$.
- ▶ ρ : ADMM penalty factor.
- ▶ $-\boldsymbol{\lambda}^T (\mathbf{x} - \mathbf{y})$: enforces the equality constraint.
- ▶ $\frac{\rho}{2} \|\mathbf{x} - \mathbf{y}\|^2$: penalizes mismatch between $\mathbf{x}$ and $\mathbf{y}$.
- ▶ The algorithm alternates between updating $\mathbf{x}$, updating $\mathbf{y}$, and updating $\boldsymbol{\lambda}$.

FaBo: ADMM Updates

$$\mathbf{x}^{k+1} = \arg \min_{\mathbf{x} \in \mathcal{M}} \mathcal{L}(\mathbf{x}, \mathbf{y}^k, \boldsymbol{\lambda}^k)$$

$$\mathbf{y}^{k+1} = \arg \min_{\mathbf{y}} \mathcal{L}(\mathbf{x}^{k+1}, \mathbf{y}, \boldsymbol{\lambda}^k)$$

$$\boldsymbol{\lambda}^{k+1} = \boldsymbol{\lambda}^k - \rho(\mathbf{x}^{k+1} - \mathbf{y}^{k+1})$$

- **x-update:** solves the convex blocklength problem under

$$\sum_i x_i \leq T_{\max}, \quad x_i \geq 0$$

- **y-update:** handles the non-convex finite-blocklength part using a proximal approximation.
- **λ -update:** increases or decreases pressure until

$$\mathbf{x}^{k+1} \approx \mathbf{y}^{k+1}$$

- x and y are updated and optimized separately for convex and non-convex parts separately but in the end they should both agree to one blocklength value n .

FaBo: How $\mathbf{x}$ and $\mathbf{y}$ are Updated

1) $\mathbf{x}$ -update: convex constrained subproblem

$$\mathbf{x}^{k+1} = \arg \min_{\mathbf{x} \in \mathcal{M}} \left[G(\mathbf{x}) - (\boldsymbol{\lambda}^k)^T (\mathbf{x} - \mathbf{y}^k) + \frac{\rho}{2} \|\mathbf{x} - \mathbf{y}^k\|^2 \right]$$

- ▶ Here $\mathbf{y}^k$ and $\boldsymbol{\lambda}^k$ are fixed.
- ▶ The update optimizes the convex part $G(\mathbf{x})$.
- ▶ Because $\mathbf{x} \in \mathcal{M}$, the paper solves this using KKT conditions.

2) $\mathbf{y}$ -update: non-convex part with gradient

$$\mathbf{y}^{k+1} = \arg \min_{\mathbf{y}} \left[\nabla H(\mathbf{x}^{k+1})^T (\mathbf{y} - \mathbf{x}^{k+1}) - (\boldsymbol{\lambda}^k)^T (\mathbf{x}^{k+1} - \mathbf{y}) + \frac{\rho + \psi}{2} \|\mathbf{x}^{k+1} - \mathbf{y}\|^2 \right]$$

$$\boxed{\mathbf{y}^{k+1} = \mathbf{x}^{k+1} - \frac{1}{\rho + \psi} \left(\nabla H(\mathbf{x}^{k+1}) + \boldsymbol{\lambda}^k \right)}$$

FaBo Results: Simulation Setup

- ▶ Number of arriving packets:

$$N_p = 10$$

- ▶ Packet size:

$$a = 50 \text{ bits}$$

- ▶ Transmit power:

$$P = 10 \text{ mW}$$

- ▶ Bandwidth:

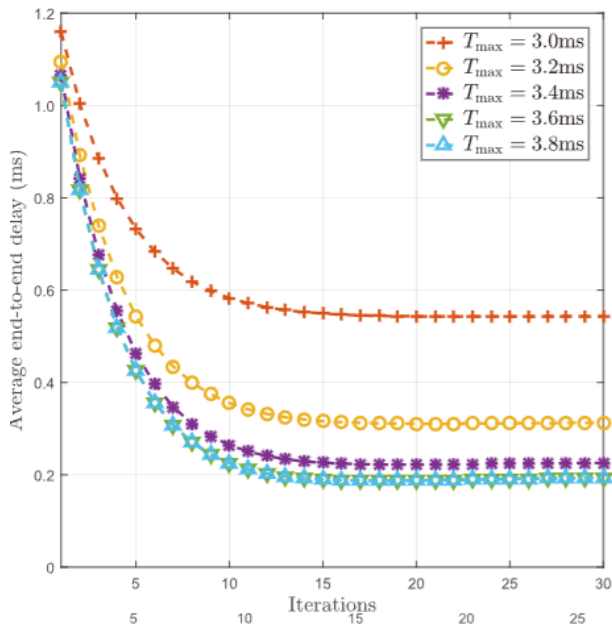
$$B = 1 \text{ MHz}$$

- ▶ Single-user channel gain:

$$|h|^2 = 1$$

- ▶ FaBo is evaluated by average end-to-end delay and convergence behavior.

FaBo Result 1: Fast Convergence



- ▶ The paper plots average end-to-end delay versus FaBo iterations.
- ▶ FaBo converges to the minimum delay in about:

15 iterations

- ▶ This means the flexible proximal ADMM update reaches a stable blocklength solution quickly.
- ▶ The result supports the claim that FaBo is suitable for low-latency systems.
- ▶ The paper also observes that as $T_{\max}$ increases:

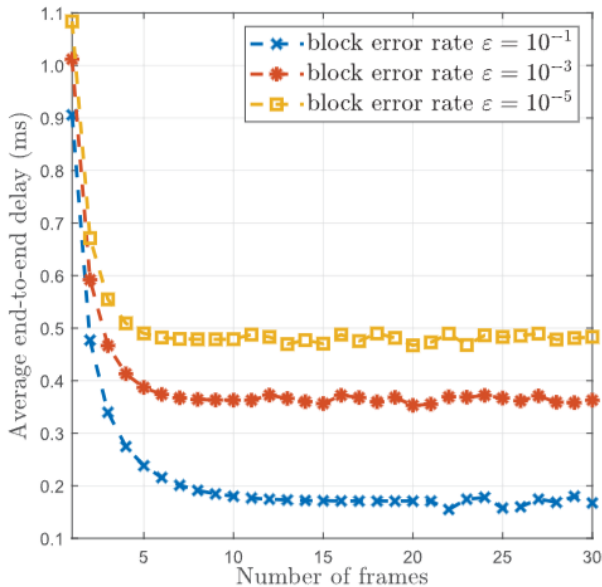
$D_{\text{aver}}^{\text{SU}}$ first decreases and then becomes stable.

- ▶ Interpretation:

too small $T_{\max} \Rightarrow$ not enough service time

sufficient $T_{\max} \Rightarrow$ delay cannot improve much further

FaBo Result 2: Choosing the Number of Frames



- ▶ The paper studies delay versus the number of frames L .
- ▶ As L increases:

D_{aver}^{su} first decreases, then converges.

- ▶ Reason:

$L \uparrow \Rightarrow$ more flexibility in choosing adaptive frame lengths

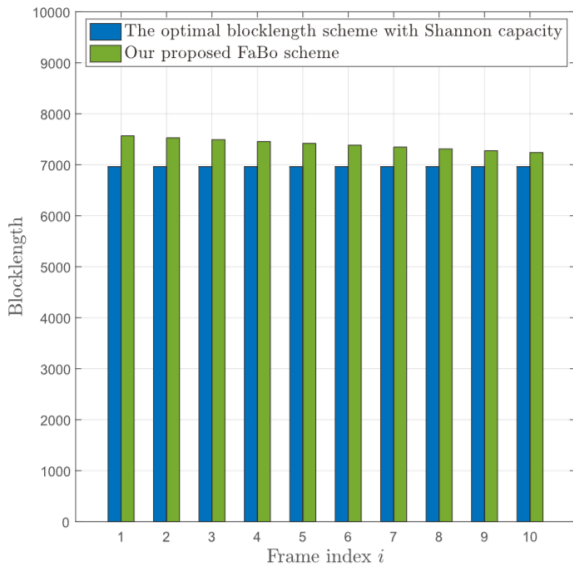
- ▶ But after a point, extra frames receive zero or negligible blocklength.
- ▶ The optimal effective number of frames is:

$L^* =$ index of the last non-zero blocklength frame

- ▶ Practical method:

choose a large L , run FaBo, then find last $n_i > 0$.

FaBo Result 3: Blocklength Pattern



- ▶ Earlier frames are longer:

large $n_i \Rightarrow$ lower queuing delay

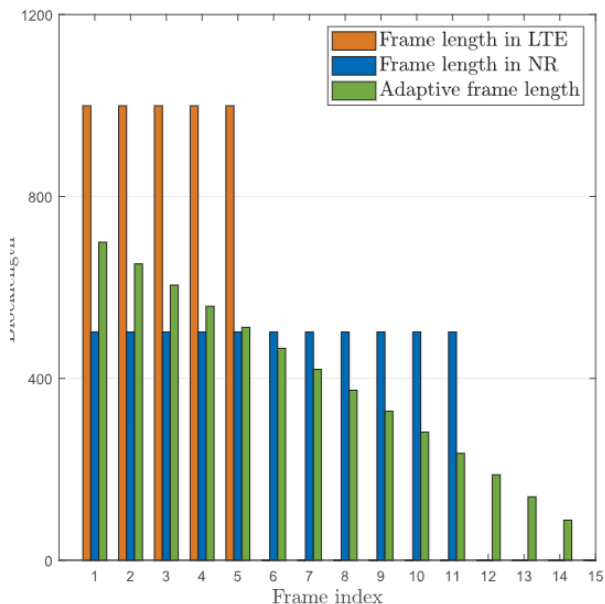
- ▶ This reduces queuing delay for early packets.
- ▶ Later frames are shorter:

small $n_i \Rightarrow$ lower transmission delay

- ▶ Therefore, FaBo balances:

D_{queue} and D_{trans}

FaBo Result 4: Comparison with 5G NR



- ▶ The paper compares FaBo with fixed-blocklength 5G NR.
- ▶ 5G NR uses fixed TTI:

$$T_{\text{NR}} = 0.5 \text{ ms}$$

- ▶ FaBo can have larger queuing delay than 5G NR:

$$D_{\text{queue}}^{\text{FaBo}} > D_{\text{queue}}^{\text{NR}}$$

- ▶ But FaBo has much smaller transmission delay:

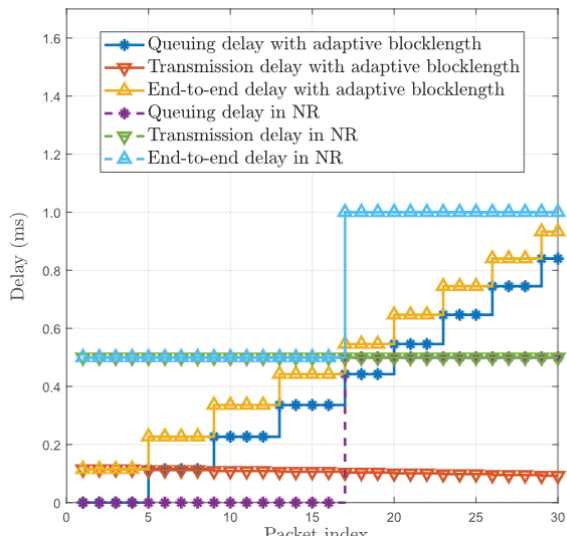
$$D_{\text{trans}}^{\text{FaBo}} < D_{\text{trans}}^{\text{NR}}$$

- ▶ Overall:

$$D_{\text{E2E}}^{\text{FaBo}} < D_{\text{E2E}}^{\text{NR}}$$

- ▶ This proves that minimizing only one delay component is not enough.

FaBo Result 5: Comparison with LTE/NR Schemes



- ▶ The paper compares FaBo against:

LTE long-frame scheme

5G NR short-frame schemes

one-packet-one-frame scheme

- ▶ LTE long frame:

$$T = 1 \text{ ms}$$

- ▶ 5G NR short frames:

$$T = 0.5, 0.25, 0.125, 0.0625 \text{ ms}$$

- ▶ FaBo achieves smaller average end-to-end delay than fixed-frame schemes.
- ▶ fixed TTI cannot adapt to packet queue and rate tradeoff
- ▶ FaBo adapts each frame length according to the delay objective.

FaBo Results: Main Inference

- ▶ FaBo converges quickly:

≈ 15 iterations

- ▶ It identifies the effective number of useful frames:

$$L^* = \text{last non-zero } n_i$$

- ▶ It produces decreasing frame lengths:

$$n_1 > n_2 > \dots$$

- ▶ Long early frames reduce queuing delay:

$$R(n_i)n_i \uparrow \Rightarrow \text{more packets served early}$$

- ▶ Short later frames reduce transmission delay.
- ▶ Final conclusion: Adaptive blocklength gives better end to end latency by better queuing-transmission delay balance than fixed LTE/NR TTI.

Comparison with our method: Paper 1

- ▶ This paper optimizes for the sum rate under latency constraints by finding the optimal precoder while we directly optimize for the latency.
- ▶ The blocklength is fixed and only the precoder is optimized.
- ▶ Paper uses lower bound for FBL Rate to find first order optimal solution this is problematic since the assumptions are too extreme (such as treating the SNR as constant when it depends on the precoder), however we keep the non-convex FBL rate and find the optimal solution by parameterizing with NN and using gradient descent.
- ▶ Only delay constrained users are modeled using FBL rate while delay tolerant users are modeled with Shannon Rate and are assumed to have infinite blocklength. We model all users with FBL rate to minimize latency.

Comparison with our method: Paper 2

- ▶ Paper assumes bits per packet based continuous transmission while we assume large payload of bits that are split between sub-block coherent channels.
- ▶ This paper does directly optimize for the latency but only by optimizing the blocklength while we optimize both precoder and blocklength to make a lower blocklength possible.
- ▶ Paper considers tradeoff with queuing delay while we dont.
- ▶ Paper assumes fixed bits per packet and optimizes blocklength only. However we opt to fit as many bits into a sub-block as possible after finding the optimal precoder and sub-blocklength. This is because we recognize that fitting max bits into initial sub-block is optimal compared to having a lower blocklength to reduce latency. This is due to the fact that even though maximizing bits and reducing symbols result in the same purpose of increasing bits/symbol and thus reducing latency they have different effects on the FBL rate inequality equation.